

research

Phase 0 — Research

Feature: Authored AI/IT Knowledge Areas · **Date:** 2026-09-07

Every decision below was settled by a measurement taken on this tree, and the measurement is quoted with it. Where a decision rests on judgement rather than measurement, that is said in the decision itself rather than dressed up.

There were **no unresolved NEEDS CLARIFICATION** entering this phase; the five open questions were closed in the spec's Clarifications section. What follows are the design decisions those clarifications did not settle.

R1 — How is the publication decision computed?

Decision: One function, `AreaPublicationOf`, consumed by every surface that needs the answer. Not a shared constant, not a shared vocabulary of reasons — a shared *decision*.

Rationale: This is the measured cause of D3, and the near-miss fix would have failed. The list handler and the detail handler each implemented the rule. The list applied four tests (evidence at build, live evidence, evidence floor, derivable title); the detail route applied those **plus three more** (review present, materials present, review not stale). Every unit test passed, because each handler was tested against its own gate. **The inconsistency is a property of the pair, and nothing asserted the pair.**

The two handlers *already shared their reason strings*. What they did not share was the **sequence of tests**. A shared vocabulary would therefore have left the defect exactly where it was.

Alternatives considered:

- *Enrich the refusal so the detail route explains itself better*. Rejected on two grounds. It preserves the falsehood in the cheapest place to read — a count of 819 is what a dashboard renders, and no detail inside a 404 corrects a number nobody re-derives. And it makes the contract's own held_back block self-contradictory: an area reported held back *and* listed as available in the same response.
- *Make the list stricter and leave the detail route alone*. Same defect, other direction; still two implementations.

Consequence, deliberately kept: a review with **no** materials row is an inconsistent *build*, not an editorial refusal — the author stage writes both in one run. It gets its own state: the detail route answers 503, the list holds it back and still answers 200. One bad build record must not take down the catalogue. The invariant survives: it is not in areas[].

R2 — What selects the published area set?

Decision: The **37 modules of the existing hand-authored lexicon** (7 tracks, 137 terms), each requiring measured passage evidence, an authored document and a publication review. **Not** an evidence-frequency ranking.

Rationale: An evidence floor was implemented and **measured, then rejected, because it made topicality worse**. At floor 2, **0 of 36** surviving titles carried an AI/ML term; at floor 1, **32 of 814** did. Frequency is not topicality. Raising the bar would have been a confident regression with a green-looking justification.

The lexicon was already in the corpus and was never wired into the publication path. Its alias patterns are written against what the transcription engines *actually emitted*, including their mis-renderings — which is why it matches real transcript text where a title-keyword search does not.

Coverage measured across all three recordings: **0 of 37** modules have zero evidence, **0 of 137** terms match zero passages. Distribution: **16** strongly evidenced (≥ 40 passages), **9** moderate (20–39), **12** thin (< 20), thinnest at exactly **1**.

Alternatives considered: mining a fresh topical list (rejected — the mined list is the defect); a topical classifier as the filter (rejected — it scores relevance of a *name*, and the names were synthesised from nulls); an evidence floor (rejected by the measurement above).

Result: published areas 819 $\rightarrow$ **42**; areas answering “not published” 817 $\rightarrow$ **3**; AI-or-IT term in name 4.5% $\rightarrow$ **64.3%**.

Honest limit: the relevance probe measures whether an area’s *name* carries a domain term. It misses real AI areas whose titles use no jargon, so 42.9% (AI/ML term) understates the change. The figure is a floor, not the value.

R3 — Why were 814 areas being published at all?

Decision: An area is published **iff its title is non-empty**.

Rationale: This rule was **already written down and never enforced**. The data model said a title is “non-empty when published”; nothing checked it. The term-extraction stage was behaving correctly under its own rule — it emits single-term proposals by design. The defect was that **nothing between the miner and the API distinguished a proposal from a publication**.

The single-word area names visitors saw were never stored. The backend **synthesised a display title from member terms when the title was null**. The symptom was a rendering of an absence.

Consequence: nothing is deleted. Withheld rows go to unpublished-areas.jsonl; the term rows (17,811) and the passage registry are untouched. The rule is implemented in **two places so a re-run reproduces it** — the policy module and the pipeline reconcile stage.

R4 — Where does authored content live, and in what format?

Decision: `workshop/curriculum/learning/NN-<slug>.json`, one `curriculum-kit Document` per authored area, decoded with `DecodeDocument`.

Rationale: `DecodeDocument` **rejects unknown fields**. This is the load-bearing property: a misspelled key otherwise drops its material *silently*, and a lesson quietly missing a diagram is indistinguishable from a lesson that never had one. Failing loudly at load is worth the strictness.

`NN-` is a sort key only. **Identity is the area’s taxonomy ULID inside the file**, so an authored document binds itself to an extracted area with no mapping table to maintain and nothing to keep in sync.

An unconfigured catalogue is a **determined** `no_learning_catalog`; an unreadable one is a 503. The distinction matters: “not configured” is a known state, “cannot read” is a fault.

Alternatives considered: a database (rejected — content is authored as committed files, and a migration adds a synchronisation problem this design does not have); Markdown with front matter (rejected — the kit’s `Document` already carries the shape, and a second parser is a second place to disagree); filename as identity (rejected — a rename would silently orphan an area).

R5 — How are video anchors produced and expressed?

Decision: URL grammar `/chapters/<slug>[/transcript][?t=<seconds>][&end=<seconds>][#p-<pid>]`, written down in `platform/frontend/docs/time-links.md` as the producer’s authority. Spans are built from engine-measured start/end times in the transcript segment sidecars, joined on passage id.

Rationale: The passage registry carries **only an end time**. The registry alone therefore cannot produce a start, and any start derived from it would be invented. Joining on `pid` against the segment sidecar is what makes the start measured.

Three producer rules, each preventing a link that *looks* fine:

- end is emitted **only when** `> t` — the documented consumer ignores a malformed span, so producing one yields a broken link that renders correctly.
- `#p-` is emitted **only for a Crockford-shaped ULID** — the grammar reserves that prefix.
- A chapter the server does not serve gets `href: null`, a stated `unresolved_reason`, its anchor intact, and a count. **No link is invented.**

Measured: 205 spans, 479 contract-conformant links across 37 documents. The 12 pre-existing documents previously carried **zero** links into any recording.

R6 — How is the assessment gated, and how are answers kept out of the wire?

Decision: Availability is decided by `AvailabilityOf` in the kit; submission by `Submit` (never the ungated `Grade`). Questions reach the wire through a purpose-built `QuestionForTaking` projection that has **no answer field at all**.

Rationale: The kit's `Question` carries `CorrectChoices/Explanation` with `omitEmpty`, so **marshalling the kit type directly would ship the key** the moment a question had one. A projection built as a map with no answer field cannot leak a future upstream field either — the protection survives changes to the type it projects from.

Keys and explanations are disclosed **only in the submit result**, where they are feedback rather than a spoiler.

The gate's load-bearing mutation is “open as soon as *any* lesson completes”. It passes five of the eight assertions and is caught by exactly one — which is why the fixture's required lesson set must be a **proper subset** of its lessons. A fixture where they coincide would let the defect through while looking thorough.

Alternatives considered: gating in the frontend (rejected — the client would still have been sent the answers); a second availability test in the consumer (rejected — that is R1's defect class in a new place).

R7 — How is learner progress held?

Decision: Session-keyed, on the server, in a file separate from the reading- position store. One Mutate under a single lock.

Rationale: A Get+Put cycle **silently drops a concurrent completion**. The two stores are kept apart because they have different shapes, different lifetimes and different costs when lost.

Session identity is accepted through **one** channel (a header). The established interviewing module accepts it three ways — body, query parameter and header — and three places to supply it are three places for them to disagree.

R8 — What is deliberately NOT ported from the interviewing module?

Recorded so absence is not read as oversight:

Not ported	Why
difficulty, category on questions	No such field exists in curriculum-kit. Requires an upstream change. Not fabricated.
Spaced repetition (streak, due_at)	The kit declines it deliberately. The interviewing module writes both and reads neither — porting a field no route consumes would add a maintenance surface with no behaviour.
Localisation	The kit is single-language by its own admission.
Plan corpus features	No plan corpus exists here.
A second slug identity	The kit mints no second identity scheme; the ULID is the identity.

And what was fixed rather than copied: the interviewing module's ? kind= filter is unvalidated and passes through to storage, so an unknown kind returns an empty list that reads as "there are none". Here an unknown kind is a **400**. Copying the behaviour would have copied a defect.

R9 — Why is curriculum-kit consumed by a replace directive?

Decision: replace to ../../../../submodules/curriculum-kit, matching the pattern the module's five existing siblings already use.

Rationale: The umbrella is the only place all of them exist together.

This is an open gap, not a resolved decision. submodules/curriculum-kit is **not a git repository** — it has no commit to pin, so a truthful helix-deps.yaml entry cannot be written, and a fabricated ref: would be a bluff. verify-standalone-clone.sh correctly went from 2 gaps to 3 and names this row. Recorded verbatim in go.mod: **inside the umbrella this builds; a standalone clone of this repository does not**. The remedy — publish it and mount it as a gitlink — is upstream and is an operator decision.